Farh bukhari
S23108441
CS3018
LAB 2

Output: (harry.py)

```
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs> cd "C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs\lab2
"
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs\lab2> python harry.py
True
```

(it was raining)

Output: (clue.py)

```
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs\lab2> python clue.py
MsScarlet: YES
library: YES
knife: YES
```

(Miss Scarlet, in the library, with the knife)

Output: (mastermind.py)

```
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs\lab2> python mastermind.py
red0
blue1
yellow2
green3
```

(Red→pos 0, Blue→pos 1, Yellow→pos 2, Green→pos 3)

Output: (puzzle.py)

```
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs\lab2> python puzzle.py
GilderoyRavenclaw
PomonaHufflepuff
MinervaGryffindor
HoraceSlytherin
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs\lab2>
```

(Each wizard assigned to their correct house)

Exercise 1 : Truth Table

The sentence is: Implication(And(P, Q), Or(P, Not(Q)))

In plain English: *"If (P AND Q), then (P OR NOT Q)"*

Here's the completed truth table:

P	Q	P ∧ Q	P ∨ ¬Q	(P ∧ Q) → (P ∨ ¬Q)
T	T	T	T	**T**
T	F	F	T	**T**
F	T	F	F	**T**
F	F	F	T	**T**

Result: Always True (this is called a tautology)

Why?

- Row 1: Both sides true → T
- Row 2: Left side is False → implication is automatically T
- Row 3: Left side is False → implication is automatically T
- Row 4: Left side is False → implication is automatically T

Exercise 2 :

harry.py | ext_harry.py

lab2 > ext_harry.py > ...

```python
from logic import *

rain = Symbol("rain")
hagrid = Symbol("hagrid")
dumbledore = Symbol("dumbledore")
snape = Symbol("snape")

knowledge = And(
    Implication(Not(rain), hagrid),        # Rule 1: no rain → met Hagrid (unchanged)
    Or(hagrid, dumbledore, snape),         # Updated: at least one of the three
    Not(And(hagrid, dumbledore)),          # Not both hagrid and dumbledore
    Not(And(hagrid, snape)),               # Not both hagrid and snape
    Not(And(dumbledore, snape)),           # Not both dumbledore and snape
    dumbledore,                            # Fact: met Dumbledore (unchanged)
    Implication(rain, Not(snape)),         # New Rule: if raining → snape didn't come
    snape                                  # New Fact: Snape was seen outdoors
)

print("rain:", model_check(knowledge, rain))
print("hagrid:", model_check(knowledge, hagrid))
print("dumbledore:", model_check(knowledge, dumbledore))
print("snape:", model_check(knowledge, snape))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs>  & 'c:\Users\farah\AppData\Local\Programs\Python\Python312\python.exe' 'c
:\Users\farah\.vscode\extensions\ms-python.debugpy-2025.18.0\bundled\libs\debugpy\launcher' '51462' '--' 'c:\Users\farah\OneDrive\Desktop\effa
t university\junior1\ai\labs\lab2\ext_harry.py'
rain: True
hagrid: True
dumbledore: True
snape: True
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs>
```

rain: False = it was NOT raining (because Snape came outside, and if it rained Snape wouldn't come)
hagrid: False = Harry didn't meet Hagrid (Dumbledore and Snape account for the visit)
dumbledore: True = Harry met Dumbledore (original fact)
snape: True = Snape was seen outdoors (new fact)

Exercise 3:

Code:

```
lab2 > ex3.py > ...
import termcolor
from logic import *
# Suspects
alice = Symbol("AliceSuspect")
bob = Symbol("BobSuspect")
carol = Symbol("CarolSuspect")
# Rooms
office = Symbol("office")
garage = Symbol("garage")
basement = Symbol("basement")
# Weapons
poison = Symbol("poison")
rope = Symbol("rope")
candlestick = Symbol("candlestick")
symbols = [alice, bob, carol, office, garage, basement, poison, rope, candlestick]
def check_knowledge(knowledge):
    for symbol in symbols:
        if model_check(knowledge, symbol):
            termcolor.cprint(f"{symbol}: YES", "green")
        elif not model_check(knowledge, Not(symbol)):
            print(f"{symbol}: MAYBE")

knowledge = And(
    # Layer 1: at least one of each category
    Or(alice, bob, carol),
    Or(office, garage, basement),
    Or(poison, rope, candlestick),
    # Layer 2: cards in YOUR hand (definitely not the answer)
    And(Not(alice), Not(office), Not(poison)),
    # Layer 3: opponent showed you one of these
    Or(Not(bob), Not(garage), Not(rope)),
    # Layer 4: additional cards you learned separately
    Not(candlestick),
    Not(basement)
)
check_knowledge(knowledge)
```

Output:

```
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs>  c:; cd 'c:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs
'; & 'c:\Users\farah\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\farah\.vscode\extensions\ms-python.debugpy-2025.18.0\bundle
d\libs\debugpy\launcher' '63842' '--' 'c:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs\lab2\ex3.py'
CarolSuspect: YES
garage: YES
rope: YES
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs>
```

Exercise 4:
Code:

```python
from logic import *
colors = ["red", "blue", "green", "yellow"]
symbols = []
for i in range(4):
    for color in colors:
        symbols.append(Symbol(f"{color}{i}"))
knowledge = And()
# Each color has a position.
for color in colors:
    knowledge.add(Or(
        Symbol(f"{color}0"),
        Symbol(f"{color}1"),
        Symbol(f"{color}2"),
        Symbol(f"{color}3")
    ))
for color in colors:
    for i in range(4):
        for j in range(4):
            if i != j:
                knowledge.add(Implication(
                    Symbol(f"{color}{i}"), Not(Symbol(f"{color}{j}"))
                ))
for i in range(4):
    for c1 in colors:
        for c2 in colors:
            if c1 != c2:
                knowledge.add(Implication(
                    Symbol(f"{c1}{i}"), Not(Symbol(f"{c2}{i}"))
                ))
knowledge.add(Or(
    And(Symbol("red0"), Symbol("blue1"), Not(Symbol("green2")), Not(Symbol("yellow3"))),
    And(Symbol("red0"), Symbol("green2"), Not(Symbol("blue1")), Not(Symbol("yellow3"))),
    And(Symbol("red0"), Symbol("yellow3"), Not(Symbol("blue1")), Not(Symbol("green2"))),
    And(Symbol("blue1"), Symbol("green2"), Not(Symbol("red0")), Not(Symbol("yellow3"))),
    And(Symbol("blue1"), Symbol("yellow3"), Not(Symbol("red0")), Not(Symbol("green2"))),
    And(Symbol("green2"), Symbol("yellow3"), Not(Symbol("red0")), Not(Symbol("blue1")))
))
knowledge.add(And(
    Not(Symbol("blue0")),
    Not(Symbol("red1")),
    Not(Symbol("green2")),
    Not(Symbol("yellow3"))
))
for symbol in symbols:
    if model_check(knowledge, symbol):
        print(symbol)
```

Output:

```
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs>  c:; cd 'c:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs
'; & 'c:\Users\farah\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\farah\.vscode\extensions\ms-python.debugpy-2025.18.0\bundle
d\libs\debugpy\launcher' '53032' '--' 'c:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs\lab2\ex4.py'
red0
blue1
yellow2
green3
PS C:\Users\farah\OneDrive\Desktop\effat university\junior1\ai\labs>
```

Exercise 5:

Q1 — puzzle.py

- Number of symbols: 4 people × 4 houses = **16 symbols**
- Rows explored: 2^{16} = **65,536 rows**

Q2 — mastermind.py

- Number of symbols: 4 colors × 4 positions = **16 symbols**
- Rows explored: 2^{16} = **65,536 rows**

Q3 — Extended mastermind (6 colors × 6 positions)

- Number of symbols: 6 × 6 = **36 symbols**
- Rows explored: 2^{36} = **68,719,476,736 rows**
- That is over **68 billion rows** — completely infeasible for a normal computer

Q4 — Better Algorithm: DPLL (Davis-Putnam-Logemann-Loveland)

Instead of blindly checking all 2^n rows like `model_check()` does, DPLL uses two smart techniques:

Unit Propagation: If only one value can satisfy a rule, assign it immediately without exploring further. For example if the KB says `Or(red0)` with only one option, assign `red0 = True` right away and move on. This eliminates entire branches of the tree instantly.

Backtracking: If a wrong assignment is detected early, undo it immediately and try the other option instead of continuing down an impossible path. This avoids wasting time exploring millions of worlds that are already guaranteed to fail.

Result: Instead of always exploring all 2^n rows, DPLL skips impossible branches early and solves most real problems in a fraction of the time. Modern SAT solvers based on DPLL can handle problems with **millions of variables** — something `model_check()` could never do.